Urban AI

Status	In progress
Owner	Lucas Dev
Completion	0.9782608695652174
Dates	@April 30, 2024 → June 16, 2024
Tasks	Setup do Projeto, Definição de arquitetura, Layouts iniciais, Autenticação de usuários, Integração com Airbnb API, Crawlers/Coleta de eventos, Dashboard com mapa, Registro de propriedades, Engine de proximidade, Agendador de notificações, Integração com serviço de e-mails, Painel de notificações, Refino UX, Testes manuais e QA, Documentação e entrega, Criando ambiente, ajuste logo , ferramenta multiligue, Verificações e Mensageria, Monitorar propriedades por coordenada de eventos, Alter table , Trazer foto da propriedade na tela de inserção de CEP, Integrações backend, Task Orquestração, Estudo de Pricing, Notificação de Proximidade de Eventos para Usuários - Backend, Urban Infra, Listagem propriedades / eventos, Back-end para menu “Eventos”, Front-end para menu “Eventos”, Salvar propriedade - Front-end, Salvar propriedade - Back-end, Erro prisma na AWS, Feature pricing - Back-end, Urban - Stripe, Empoli - String, Deploy Urban / Bug, Urban - Front Mapa interativo (mock), Esquecer senha e confirmação, Fazer pull request feat/mailsender, Alterar rário padrão de 25 para 30, Excluir propriedades, Refatoração tela settings, Feature - Aceite automático, Resolver conflito no Github - Urban AI, Vídeo e ajuste Connect - Design
Collaborators	Marcos Gabriel Jefferson Severiano Lucas Dev Lumina Lab

About this project

Sistema de Monitoramento de Eventos e Integração com Airbnb

1. Nome do Projeto

Desenvolvimento de Sistema Web para Monitoramento de Eventos em São Paulo com Integração à API do Airbnb chamado URBAN AI.

2. Objetivo Geral

Desenvolver uma plataforma web que monitore eventos na cidade de São Paulo e integre-se à API do Airbnb via serviço terceirizado, permitindo que anfitriões recebam notificações com antecedência sobre eventos próximos às suas propriedades, possibilitando ajustes estratégicos nos preços de locação.

3. Escopo Funcional

3.1. Monitoramento de Eventos

- Coleta automática de informações públicas sobre eventos (shows, festivais, jogos, conferências, etc.).
- Registro da data, local e tipo de evento em banco de dados.

3.2. Integração com Airbnb

- Conexão com a API do Airbnb via RapidAPI, assinatura a ser feita pelo contratante.
- Coleta de informações sobre as propriedades cadastradas por cada usuário.

3.3. Geolocalização e Mapeamento

- Análise de proximidade entre os eventos cadastrados e as propriedades dos usuários.

- Definição de raio de influência para cada propriedade.

3.4. Notificações Inteligentes

- Envio de alertas automáticos sobre eventos próximos com no mínimo 30 dias de antecedência.
- Central de notificações na plataforma + envio por e-mail.

3.5. Plataforma Web (SaaS)

- Interface web responsiva para gestão de propriedades e eventos.
- Acesso via login e senha para usuários cadastrados.
- Painel administrativo para gestão dos dados e usuários.

4. Tecnologias a Serem Utilizadas

- **Frontend:** React.js ou Vue.js, podendo ser ajustada pelo time técnico caso haja necessidade.
- **Backend:** Node.js ou Python (Django ou Flask), podendo ser ajustada pelo time técnico caso haja necessidade.
- **Banco de Dados:** PostgreSQL ou MongoDB, podendo ser ajustada pelo time técnico caso haja necessidade.
- **APIs:** RapidAPI (Airbnb)
- **Envio de e-mails:** SendGrid, a ser assinado pelo contratante.
- **Infraestrutura:** Servidores em nuvem (AWS, DigitalOcean ou similar), a ser custeado pelo contratante.

5. Equipe Técnica

- 1 Product Owner (responsável pela comunicação com o cliente e validação de entregas)
- 2 Desenvolvedores Sêniores
- 2 Desenvolvedores Plenos
- 1 Profissional de QA (Quality Assurance)
- 1 UX Designer

6. Prazo de Entrega

- Desenvolvimento e testes: **3 a 4 meses**

7. Implantação e Suporte

- Entrega do sistema funcional com documentação básica.
- Suporte técnico por 30 dias após a entrega.
- Possibilidade de contrato adicional de suporte e melhorias contínuas.

8. Custos Adicionais de Responsabilidade do Cliente

- **Assinatura do RapidAPI:** estimativa de até **US$ 200/mês**
- **Serviço de envio de e-mails (SendGrid ou similar):** valor variável conforme volume
- **Infraestrutura de servidores:** média estimada de **R$ 2.000,00/mês**
- **Integração com meio de pagamento:** contratação opcional pelo cliente

9. Entregáveis

- Plataforma web funcional e hospedada
- Integração com a API do Airbnb
- Painel administrativo
- Sistema de notificações por e-mail e painel
- Manual básico de uso

10. Fora do Escopo

- Desenvolvimento de aplicativos móveis
- Suporte a cidades além de São Paulo na versão inicial
- Personalização individual por usuário fora do plano padrão

Brandbook e artes do projeto

https://drive.google.com/drive/folders/1z3vrd5-f5xrVz7szQf_yzIRVWVx8teNU?usp=sharing

UI das telas do projeto

https://www.figma.com/design/4as0jz5gbtv3TErDNz3PWe/Urban-AI?node-id=0-1&t=4RhFcrMMWfZKBsR1-1

Semanais:

24/04 - Introdução e kickoff do projeto;

02/05 - Apresentação do refino de UX e alinhamento de atividades;

09/05 - Entrega das primeiras telas, https://urban-front-develop-719774307855.us-central1.run.app/app

16/05 - Alinhamento sobre APIs e integrações com ticketerias

Input do cliente: Adicionar Ticket360

06/06 - Acompanhamento do frontend com validação do cliente, com link do vercel para validação

13/06 - Acompanhamento do frontend com validação do cliente

27/06 - Apresentação do fluxo com aceites de sugestões

04/07 - Liberação para testes com remoção da GCP

18/07 - Subida do link na railway ara testes dos fluxos de cadastro de usuários e propriedades, e ajustes para importação a partir de anuncios e link do editor.

08/08 - Inicio desenvolvimento de fluxo de mensagens de email e Deashboards.

15/08 - Decisão interna de melhoria de front e comunicação ao cliente, sem ressalvas.

22/08 - Subida para prod e inicio de testes.

29/08 - Ultima reunião formal e feedback de testes.

Set - Ajustes, correções e melhorias solicitadas pelo cliente:

Aumento do raio de busca

Melhorias na tela de importação de propriedades

Notificação em lote.

Aceite automático de propriedades.

Out - Ajustes Rapid API e reuniões comerciais.

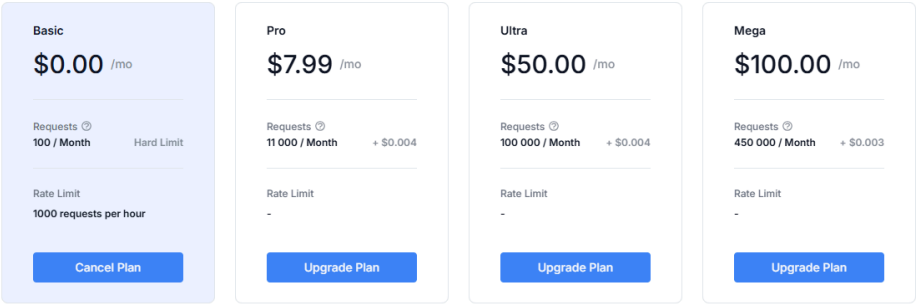
Conexão Airbnb

API utilizada: https://rapidapi.com/apiheya/api/airbnb45

Overview:

Locate Airbnb rooms using geo coordinates or location. Find out room availability and prices for a location (such as New York) between two dates. Search for rooms in an area limited by two geographical points between two dates. Locating location names can be made easier with autocomplete.

Custos:



Estudo ticketerias

Plataforma	Foco Principal	Porte de Evento	Atuação em SP
Ticket360	Casas de show paulistanas	Médio a grande	Muito alta
Sympla	Geral / Independentes	Pequeno a médio	Alta
Eventim	Shows e grandes eventos	Grande	Alta
Ingresse	Entretenimento jovem	Médio a grande	Alta
Ticketmaster	Grandes produções	Grande	Alta
Even3	Acadêmico/corporativo	Pequeno a grande	Média
BlueTicket	Diversos	Pequeno a médio	Média

Project tasks

Plataforma	API Pública	Lista Eventos?	Observações
Ticket360	❌ Não	❌ Não	Pode ter API privada sob NDA
Sympla	✅ Sim	✅ Sim	Melhor opção para uso imediato
Eventim	❌ Não	❌ Não	Acesso restrito via negociação
Ingresse	⚠️ Parcial	✅ Sim (via parceria)	Precisa de acordo
Ticketmaster	⚠️ Global	✅ Sim (lá fora)	No Brasil, acesso depende da T4F
Even3	✅ Sim	✅ Sim	Boa para eventos corporativos/acadêmicos

Onboarding Airbnb

Através do link do profile, o sistema realiza a consulta via API para obter todas as propriedades associadas ao perfil e as lista para o usuário, que deve marcar as propriedades que ele deseja monitorar;

OBS: O AirBnb só compartilha a localização exata mediante a reserva, por isso é necessário um passo adicional onde o usuário informa o CEP da propriedade para que possamos armazenar no banco de dados do sistema para posterior cruzamento com dados de Evento.

attachment:fdad2e79-e51c-4c2d-905a-ccff7312401d:Onboarding_AirBnB.mov

Coleta de eventos

Nossa estratégia é começar a coleta de eventos seguindo dois critérios de prioridade:

1. Disponibilidade de API
2. Relevância da Ticketeira

Primeira integração, Sympla: https://developers.sympla.com.br/api-doc/

API Even3: https://docs.even3.com.br/

OBS: Após a conclusão da primeira integração, podemos começar a desenvolver a Engine de proximidade em paralelo.

Resultado: A estratégia via API não deu certo, devido a impossibilidade de listar eventos, seguiremos por Web Crawler.

🎯 Sistema 100% Google Maps - Passo a Passo:

1. 🔧 Setup inicial :

```
sql
-- Adicionar colunas na tabela addresses
ALTER TABLE addresses
ADD COLUMN latitude DECIMAL(10, 8) NULL,
ADD COLUMN longitude DECIMAL(11, 8) NULL,
ADD COLUMN geocoding_status VARCHAR(50) DEFAULT 'pending',
ADD COLUMN geocoding_date TIMESTAMP NULL,
ADD COLUMN geocoding_accuracy VARCHAR(50) NULL;
```

2. 🔑 Configurar API Key:

```
javascript
// Cole sua API key do Google Maps
const GOOGLE_MAPS_API_KEY = 'sua-api-key-aqui';
```

3. 🚀 Executar tudo de uma vez:

```
javascript
// Executa o fluxo completo automaticamente
await executeFullWorkflow();
```

📊 O que o sistema faz:

✅ **Geocodifica todos os endereços** da tabela `addresses`

✅ **Controla rate limit** (20 req/segundo)

✅ **Retry automático** para falhas temporárias

✅ **Relatórios detalhados** de progresso

✅ **Calcula proximidades** entre eventos e endereços

✅ **Status tracking** (pending/success/failed)

💰 Custos transparentes:

- **Mostra custo em tempo real** durante processamento
- **$5 por 1.000 geocodificações** após os $200 gratuitos
- **Exemplo:** 10.000 endereços = $0 (dentro do limite gratuito)

⏱️ Performance:

- **20 requisições/segundo** (dentro do limite Google)
- **~1.000 endereços/hora**
- **Rate limiting inteligente** para evitar bloqueios

🎯 Resultado final:

Você terá:

1. ✅ Todos endereços com latitude/longitude
2. ✅ Lista de proximidades evento ↔ endereço
3. ✅ Relatórios de qualidade e custos
4. ✅ Sistema pronto para consultas de distância

Quantos endereços você tem na tabela `addresses`? Com essa informação eu calculo o custo exato! 💡

❌ **NÃO vai funcionar** se os eventos também não têm coordenadas!

Para calcular proximidades, **AMBAS** as tabelas precisam ter latitude/longitude. Vou te mostrar como resolver:

Geocodificar EVENTOS e ENDEREÇOS - Sistema CompletoCódigo // 🚀 GEOCODIFICAR AMBAS TABELAS: EVENTS + ADDRESSES
// Para quando NENHUMA das duas tem coordenadas

// ==========
// 🔧 1. CONFIGURAÇÃO E SETUP
// ==========

const GOOGLE_MAPS_API_KEY = 'SUA_AP

⚠️ Situação atual vs Solução:

❌ Problema:

- **Events:** Só têm `enderecoCompleto` (sem lat/lng)
- **Addresses:** Só têm `cep`, `logradouro`, etc. (sem lat/lng)
- **Proximidades:** IMPOSSÍVEL calcular sem coordenadas

✅ Solução - Geocodificar AMBAS tabelas:

```sql
-- 1. Preparar events
ALTER TABLE events
ADD COLUMN latitude DECIMAL(10, 8) NULL,
ADD COLUMN longitude DECIMAL(11, 8) NULL;

-- 2. Preparar addresses (você já sabe)
ALTER TABLE addresses
ADD COLUMN latitude DECIMAL(10, 8) NULL,
ADD COLUMN longitude DECIMAL(11, 8) NULL;
```

🚀 Fluxo completo:

```javascript
// Executa tudo automaticamente:
await geocodeEverythingAndCalculateProximities();
```

Isso vai:

1. ✅ Geocodificar **TODOS os eventos** usando `enderecoCompleto`
2. ✅ Geocodificar **TODOS os endereços** usando `logradouro + número + bairro + cidade`
3. ✅ Calcular proximidades entre eventos ↔ endereços
4. ✅ Mostrar relatório completo com custos

💰 Estimativa de custo:

Se você tem:

- **1.000 eventos** = $5
- **5.000 endereços** = $25
- **Total** = $30 (ainda dentro dos $200 gratuitos!)

📊 Resultado final:

Você terá uma **lista completa** de proximidades como:

```
Evento: "TOKENNATION EXPERIENCE" ↔ Endereço: "Rua Augusta, 123" (1.2km)
Evento: "TEDx São Paulo" ↔ Endereço: "Av. Paulista, 456" (0.8km)
```

Vou criar um endpoint mais detalhado que:

1. ✅ **Recebe endereço específico e IDs**
2. ✅ **Separa modalidades** (Metro/Trem/Ônibus individuais)
3. ✅ **Mostra estações próximas**
4. ✅ **Tempos específicos por modal**

Como testar no Insomnia:

1. Análise detalhada com IDs específicos:

```
GET http://localhost:3000/connect/transport/detailed/00bc93c6-4549-11f0-84fc-42010a400016/
12062c6d-f21d-494a-94f8-f88df6b2f662
```

2. Teste com endereços customizados (POST):

```
POST http://localhost:3000/connect/transport/custom-addresses
Content-Type: application/json

{
  "enderecoOrigem": "Club Hotel Cambridge, Rua João Adolfo, 126, Centro, São Paulo - SP",
  "enderecoDestino": "Rua Augusta, 500, Consolação, São Paulo - SP",
  "eventoId": "00bc93c6-4549-11f0-84fc-42010a400016",
  "propriedadeId": "12062c6d-f21d-494a-94f8-f88df6b2f662"
}
```

3. Teste rápido via URL:

```
GET http://localhost:3000/connect/transport/test-sao-paulo/Club%20Hotel%20Cambridge%2C%20C
entro%2C%20São%20Paulo/Rua%20Augusta%2C%20500%2C%20Consolação%2C%20São%20Paulo
```

Resposta esperada (modalidades separadas):

```
json
{
  "origem": "Club Hotel Cambridge, Centro, São Paulo",
  "destino": "Rua Augusta, 500, Consolação, São Paulo",
  "eventoId": "00bc93c6-4549-11f0-84fc-42010a400016",
  "propriedadeId": "12062c6d-f21d-494a-94f8-f88df6b2f662",
  "transportePublico": {
    "geral": {
      "duracao": "15 min",
      "distancia": "2.1 km",
      "observacao": "Melhor combinação de metro, trem e ônibus"
    },
    "metro": {
      "duracao": "12 min",
      "modalidade": "Metro",
      "icon": "🚇",
      "linhasPossiveis": ["Linha 1-Azul (Sé, República)", "Linha 2-Verde (Consolação)"]
    },
    "trem": {
      "status": "INDISPONIVEL",
      "modalidade": "Trem (CPTM)",
      "motivo": "Sem estações de trem conectando estes pontos"
    },
    "onibus": {
      "duracao": "18 min",
      "modalidade": "Ônibus",
      "icon": "🚌",
      "linhasPossiveis": "Linhas municipais SPTrans"
```

```
    }
  },
  "outrasOpcoes": {
    "carro": {
      "duracao": "8 min",
      "duracaoComTransito": {
        "texto": "12 min",
        "minutos": 12
      }
    },
    "caminhada": {
      "duracao": "25 min",
      "viavel": true
    }
  },
  "estacoesProximas": {
    "origem": [
      {
        "nome": "Sé",
        "tipo": "Metro",
        "distancia": "400m",
        "linhas": ["1-Azul", "3-Vermelha"]
      }
    ],
    "destino": [
      {
        "nome": "Consolação",
        "tipo": "Metro",
        "distancia": "300m",
        "linhas": ["2-Verde"]
      }
    ]
  },
  "recomendacao": {
    "maisRapido": "carro",
    "melhorSustentavel": "metro",
    "todasOpcoes": ["carro: 8min", "metro: 12min", "onibus: 18min"]
  }
}
```

Agora você tem:
✅ **Modalidades separadas** (Metro/Trem/Ônibus individuais)
✅ **IDs específicos** (evento e propriedade)
✅ **Estações próximas** com distâncias
✅ **Linhas específicas** do metro/CPTM
✅ **Múltiplas formas de testar**

✅ O que você quer prever exatamente?

> "Quão perto (em tempo ou distância real) estará o endereço A do endereço B no dia 23/07 às 17h, levando em conta o trânsito?"
>
> - Ocorrência de engarrafamentos, interdições, etc.

Elementos que o modelo pode considerar:

1. **Geolocalização estática**: latitude/longitude dos endereços. ✅
2. **Dados históricos de tráfego**: ✅

- Velocidade média nas vias por dia/hora da semana.

Modelo preditivo de proximidade

1. **Calendário e sazonalidade**:
   - Feriados, férias escolares, eventos locais (shows, jogos). ✅
2. **Previsão meteorológica (opcional)**:
   - Clima pode impactar o trânsito (chuvas, neblina).
3. **Tipo de deslocamento**:
   - A pé, carro, transporte público, bicicleta. ✅

Como seria o modelo?

- **Modelos estatísticos + aprendizado de máquina** para prever **tempo de deslocamento** com base em dados históricos.
- **Redes neurais recorrentes (RNN/LSTM)** ou modelos baseados em séries temporais para captar padrões de tráfego ao longo do tempo.

O que pode ser a "proximidade real"?

- **Tempo de deslocamento estimado**.
- **Distância real com base na rota**.
- **Probabilidade de chegar em X minutos**.
- Classe de acessibilidade (ex: "muito próximo", "razoável", "distante").

Ferramentas e fontes úteis:

- **Google Maps API** (com previsão de trânsito futuro).
- Frameworks como **scikit-learn**, **XGBoost**, **LightGBM** para modelos preditivos.
- Banco de dados de séries temporais.

Resumo:

Sim, é possível prever o quão próximo dois endereços estarão em uma data e horário futuros, mas a qualidade da previsão dependerá de:

- Acesso a dados históricos de tráfego confiáveis.
- Precisão do modelo e da granularidade dos dados.
- Definição clara do que significa “proximidade” no seu contexto.

4. Modelo preditivo (com API do Google Maps):

```
python
CopiarEditar
import googlemaps
from datetime import datetime

gmaps = googlemaps.Client(key="SUA_API_KEY")

def tempo_google(origem, destino, data_hora):
    timestamp = int(pd.to_datetime(data_hora).timestamp())
    result = gmaps.distance_matrix(origem, destino, mode="driving", departure_time=timesta
mp)
```

```
    return result["rows"][0]["elements"][0]["duration_in_traffic"]["text"]

print(tempo_google("Av. Paulista, 1000", "Rua Funchal, 300", "2025-07-22 18:30:00"))

A seguir um exemplo mínimo de como seria o serviço e controller em NestJS usando o pacote
oficial @googlemaps/google-maps-services-js:

ts
Copiar
Editar
// src/traffic/traffic.module.ts
import { Module } from '@nestjs/common';
import { TrafficService } from './traffic.service';
import { TrafficController } from './traffic.controller';

@Module({
  providers: [TrafficService],
  controllers: [TrafficController],
})
export class TrafficModule {}
ts
Copiar
Editar
// src/traffic/dto/predict-proximity.dto.ts
import { IsString, IsISO8601, IsIn } from 'class-validator';

export class PredictProximityDto {
  @IsString()
  origin: string;     // ex: "Av. Paulista, 1000"

  @IsString()
  destination: string; // ex: "Rua Funchal, 300"

  @IsISO8601()
  datetime: string;    // ex: "2025-07-23T17:00:00"

  @IsIn(['driving','transit','walking','bicycling'])
  mode: 'driving' | 'transit' | 'walking' | 'bicycling';
}
ts
Copiar
Editar
// src/traffic/traffic.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { Client, TravelMode } from '@googlemaps/google-maps-services-js';
import { PredictProximityDto } from './dto/predict-proximity.dto';

@Injectable()
export class TrafficService {
  private readonly logger = new Logger(TrafficService.name);
  private readonly client = new Client({});

  /**
   * Retorna distância e tempo previsto (com trânsito) entre dois pontos num datetime futu
ro.
   */
  async predictProximity(dto: PredictProximityDto) {
    const { origin, destination, datetime, mode } = dto;
```

```
    const departureTs = Math.floor(new Date(datetime).getTime() / 1000);

    this.logger.log(`Consultando Google Maps: ${origin} → ${destination} em ${datetime}`);

    const res = await this.client.distancematrix({
      params: {
        origins:      [origin],
        destinations: [destination],
        mode:         mode as TravelMode,
        departure_time: departureTs,
        traffic_model:  'best_guess',     // “pessimistic” ou “optimistic” também disponív
eis
        key:            process.env.GOOGLE_MAPS_API_KEY,
      },
    });

    const elem = res.data.rows[0]?.elements[0];
    if (elem.status !== 'OK') {
      throw new Error(`Google Maps retornou status ${elem.status}`);
    }

    return {
      distanceMeters:         elem.distance.value,
      durationSeconds:        elem.duration.value,
      durationInTrafficSecs:  elem.duration_in_traffic.value,
      trafficModel:           'best_guess',
      fetchedAt:              new Date().toISOString(),
    };
  }
}
ts
Copiar
Editar
// src/traffic/traffic.controller.ts
import { Controller, Post, Body } from '@nestjs/common';
import { TrafficService } from './traffic.service';
import { PredictProximityDto } from './dto/predict-proximity.dto';

@Controller('traffic')
export class TrafficController {
  constructor(private readonly traffic: TrafficService) {}

  @Post('predict')
  async predict(@Body() dto: PredictProximityDto) {
    return this.traffic.predictProximity(dto);
  }
}
Como usar
bash
Copiar
Editar
# 1) Instale o client do Google Maps
npm install @googlemaps/google-maps-services-js

# 2) Defina sua chave no .env
GOOGLE_MAPS_API_KEY=SEU_TOKEN

# 3) Faça um POST em /traffic/predict com JSON:
```

```
{
  "origin":      "Av. Paulista, 1000",
  "destination": "Rua Funchal, 300",
  "datetime":    "2025-07-23T17:00:00",
  "mode":        "driving"
}
```

- **Sympla** (via scraping de páginas HTML e paginação explícita)
- **Ingresse** (via chamada direta à API pública de eventos)

O resultado final são dois arquivos CSV (`events.csv` e `ingresse_events.csv`), contendo dados normalizados prontos para serem consumidos por sistemas de BI, bancos de dados ou outras aplicações.

---

2. Tecnologias e Dependências

- **Node.js** (>= v14)
- **Puppeteer**: navegador headless para interação com páginas web
- **fs** (File System): para leitura e escrita de arquivos CSV
- Flags recomendadas de execução:
  - `-no-sandbox` e `-disable-setuid-sandbox` (produção em container/CI)
  - `-disable-http2` (apenas no scraper do Ingresse, forçando HTTP/1.1)
- **Linters** e **formatadores** (opcionais, não mostrados no código-fonte)

---

3. Arquitetura do Serviço

```
bash
CopiarEditar
src/
├── sympla-scraper.ts       # Implementa `buscarEventos()` para Sympla
├── ingresse-scraper.ts     # Implementa `scrapeEvents()` + `buscarEventos()` para Ingress
e
└── index.ts                # Ponto de entrada (opcional), orquestra ambos os scrapers
```

Cada módulo exporta uma função assíncrona `buscarEventos()` que:

1. Inicializa o browser/headless
2. Navega ou faz a requisição à API
3. Extrai e normaliza os campos desejados
4. Monta o CSV final e o persiste em disco
5. Fecha o browser

---

4. Descrição dos Scrapers

4.1. Sympla (página HTML + paginação)

- **URL alvo**: `https://www.sympla.com.br/eventos/sao-paulo-sp/em-alta`
- **Seletores principais**:
  - Cartões de evento: `a.sympla-card[data-name]`
  - Nome do evento: atributo `data-name`

    - Cidade: `.pn67h1c`
    - Data: `.qtfy415`, `.sc-1bker4h-4` ou `.sta-event-week-date-text`
    - Imagem: `<img>` dentro do cartão
- **Fluxo de paginação**:
    1. Enquanto existir botão **Próximo** habilitado
    2. Clicar no botão e aguardar mudança no primeiro cartão (título, href ou contagem)
    3. Fallback: em caso de timeout, recarregar a página e tentar novamente
    4. Limite de segurança: interrompe após N páginas (ex.: 226)
- **Campos extraídos**:
    - `nome`
    - `cidade`
    - `dataInicio` e `dataFim` (quando houver intervalo "a")
    - `imageUrl`
    - `link` (href completo do cartão)
- **Saída**: arquivo `events.csv` com cabeçalho

```
csv
CopiarEditar
nome,cidade,dataInicio,dataFim,imageUrl,link
```

- **Logs**:
    - Início e fim do scraping
    - Número de eventos por página
    - Possíveis banners de cookies detectados e fechados
    - Avisos de timeout e recarregamento forçado

4.2. Ingresse (API JSON)

- **Endpoint**:

```
arduino
CopiarEditar
https://api-site.ingresse.com/events/search
  ?company_id=1
  &iso_code=BRA-SP
  &size=<tamanho_página>
  &offset=<página_atual * tamanho_página>
  &order_by_date=true
```

- **Cabeçalhos e User-Agent**:
    - `accept-language: pt-BR,pt;q=0.9`
    - User-Agent de desktop (`Mozilla/5.0 (Windows NT 10.0; Win64; x64)`)
- **Paginação**:
    - O total de páginas é obtido em `data.pagination.total_pages`
    - Itera até `pageNumber <= totalPages`
- **Campos extraídos**:

- `id`
- `title` (título do evento)
- `street` (endereço de rua ou "Endereço não disponível")
- `dateTime` (campo `session.dateTime`)
- `imageUrl` (`poster.small`)

- **Saída**: arquivo `ingresse_events.csv` com cabeçalho

```
c
CopiarEditar
id,title,street,dateTime,imageUrl
```

- **Tratamento de erros**:
  - Se `data.events` não existir, interrompe o loop e emite erro no console
  - Garante fechamento do browser em bloco `finally`

---

5. Exemplo de Uso

1. Instalar dependências:

```
bash
CopiarEditar
npm install puppeteer fs
```

2. Executar cada scraper individualmente:

```
bash
CopiarEditar
node sympla-scraper.js    # gera events.csv
node ingresse-scraper.js  # gera ingresse_events.csv
```

3. Validar saída em `./events.csv` e `./ingresse_events.csv`

---

6. Observações Finais

- **Modularização**: é possível unificar ambos os scrapers em uma única CLI, recebendo parâmetros de fonte (Sympla ou Ingresse).
- **Escalabilidade**: o uso de Puppeteer headless pode ser pesado; para grandes volumes, considere soluções baseadas em chamadas diretas a APIs ou filas de processamento.
- **Manutenção**: seletores CSS podem mudar a qualquer momento; cada alteração no layout de origem exigirá atualização dos seletores no código.

Principais mudanças

1. **Limpeza e organização de imports**
   - Remoção de declarações duplicadas de `'use client'` e de módulos (React, Hooks, componentes do Chakra UI).
   - Agrupamento único de cada componente de UI (`Badge`, `Box`, `Button`, etc.), evitando importações redundantes e facilitando a manutenção.
2. **Padronização de tipos e interfaces**
   - Unificação das definições de `EventItem` (em Dashboard e Event Log) para evitar conflitos de tipagem.

- Import único do tipo `DateRange` e do componente `DayPicker`, corrigindo duplicações.

3. **Melhorias de UX e acessibilidade**
   - Ajuste na configuração de **line-height** para facilitar leitura: agora controlável via menu de configurações (editor ou estilo global).
   - Garantia de responsividade usando `useBreakpointValue` onde necessário (página de preços).
4. **Performance e estabilidade**
   - Padronização do carregamento do date-picker e formatação de datas com `date-fns`, evitando múltiplas instâncias.
   - Eliminação de renderizações desnecessárias (imports duplicados) que poderiam inflar o bundle.

Detalhes por página

1. Dashboard (`src/app/dashboard/page.tsx`)

- Removidos 18 linhas de imports repetidos e corrigidos 17 linhas de adição para manter apenas um bloco de import.
- Consolidado `React`, `useState`, componentes do Chakra e `DayPicker` em um único import cada.

2. Event Log (`src/app/event-log/page.tsx`)

- Limpeza de 11 linhas de imports duplicados; adição de 8 linhas para corrigir ordem e eliminar redundâncias.
- Interfaces e hooks (`useState`, `format`, `DayPicker`) agora importados apenas uma vez.

3. Price (`src/app/price/page.tsx`)

- Eliminados 8 linhas de imports repetidos; reorganizadas 6 linhas de novos imports para manter consistência.
- Uso correto de `Card`, `Stack`, `useBreakpointValue` e demais componentes do Chakra UI sem repetições.

Benefícios entregues

- **Código mais enxuto**: facilita futuras manutenções e onboard de novos desenvolvedores.
- **Menor bundle**: ao evitar duplicações, reduzimos o tamanho final do JavaScript enviado ao cliente.
- **Melhor experiência de leitura**: controle de espaçamento entre linhas (line-height) ajustável.
- **Uniformidade visual**: uso consistente dos componentes do Chakra UI e formatação de datas.

Resumo das Atividades Realizadas

1. Nova Tela de Pagamento

Implementamos uma nova tela de login com suporte a múltiplos idiomas (i18n com `react-i18next`) e integração com autenticação via Google (`next-auth`). O fluxo foi ajustado para ocultar a senha durante digitação, com feedback de erro em três idiomas (português, inglês, espanhol). A interface foi criada com Chakra UI e framer-motion, melhorando a experiência visual e responsiva.

2. Integração com envio de e-mails (SendGrid)

Criamos uma estrutura completa para envio de e-mails com template dinâmico via SendGrid. Os dados do destinatário são preenchidos dinamicamente com nome, assunto e quantidade de eventos. O envio é disparado automaticamente após análise de eventos únicos não enviados.

- Adicionamos um endpoint POST `/email` para envio individual.
- Adicionamos um endpoint GET `/email` que compila eventos únicos por usuário e dispara e-mails personalizados.

3. Monitoramento de Processos

Implementamos uma estrutura para acompanhar o status de processos assíncronos:

- Criação da entidade `ProcessStatus` com estados: `running`, `completed`, `error`.
- Criamos endpoints (`GET` e `PUT` em `/process-status`) para obter e atualizar o estado do processo.
- Adicionado serviço que garante a existência de um único registro e mantém o status atualizado.

4. Adições ao `.env`

Incluímos chaves de API adicionais no `.env`, como:

- `SENDGRID_API_KEY` para e-mails
- `HERE_API_KEY` para cálculo de rotas

5. Utilitários para Cálculo de Distância

Adicionamos funções auxiliares em `src/util.ts`:

- `calculateDistance`: cálculo de distância haversine.
- `calculateDistanceHere`: integração com a API HERE para obter distância real e tempo de viagem.
- `aproximadamenteOuMenor`: comparação com margem de tolerância para validações geográficas.

6. Ajustes no sistema de tradução

Atualizamos os arquivos `common.json` para os idiomas `pt`, `en` e `es`, incluindo as traduções para as mensagens de erro e filtros de eventos.

1. Nova Tela de Pagamento e Login

- Criada uma nova tela com suporte a múltiplos idiomas utilizando `react-i18next`.
- Layout responsivo com Chakra UI e animações com Framer Motion.
- Implementado login com e-mail/senha e com Google (`next-auth`), com tratamento de erro multilíngue (pt, en, es).
- Senha protegida com input tipo `password`.

2. Envio de E-mails Automatizados (SendGrid)

- Criada integração com a API do SendGrid para envio de e-mails personalizados com template dinâmico.
- Adicionados endpoints:
  - `POST /email`: envio individual de e-mails com dados como nome, assunto e quantidade de eventos.
  - `GET /email`: compila eventos únicos por usuário e envia e-mail automático para cada um.
- Inclusão da chave `SENDGRID_API_KEY` no `.env`.

3. Monitoramento de Processos Assíncronos

- Criada a entidade `ProcessStatus` com estados `running`, `completed`, `error`.
- Adicionados endpoints:
  - `GET /process-status`: consulta do status atual.
  - `PUT /process-status`: atualização manual do status.
- Implementado serviço `ProcessService` com verificação de existência e atualização segura do registro.

4. Cálculo de Distância com HERE API

- Inclusão da chave `HERE_API_KEY` no `.env`.
- Criadas funções utilitárias em `src/util.ts`:
  - `calculateDistance`: cálculo haversine.
  - `calculateDistanceHere`: tempo e distância real com HERE Maps API.
  - `aproximadamenteOuMenor`: função de validação com tolerância.

5. Sistema de Tradução Aprimorado

- Expandido o suporte multilíngue em `common.json` para português, inglês e espanhol.
- Adicionadas traduções de headers, menus, eventos e mensagens de erro nos arquivos `public/locales/{pt,en,es}/common.json`.
- Ajustes visuais em componentes como `Header`, `Tabs`, `Alert`, `Inputs` e mensagens como “Nenhum evento aqui”.

6. Melhorias em Componentes e Páginas

- Adição do componente `Header` em várias páginas como `/dashboard`, `/event-log`, `/near-events`, `/pricing`.
- Navegação atualizada com links para `/dashboard`, `/near-events`, `/pricing`.
- Filtros de eventos (Todos, Urgentes, Passados) aplicados com tabs dinâmicas.
- Ajustado o logout para limpar o token e redirecionar corretamente.

Project tasks

Tasks

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
Task Orquestração								Urban AI
Documentação e entrega								Urban AI
Resolver conflito no Github - Urban AI	Lucas Dev					@October 10, 2025		Urban AI
Feature - Aceite automático	Lucas Dev			100		@October 9, 2025		Urban AI
Trazer foto da propriedade na tela de inserção de CEP								Urban AI
Vídeo e ajuste Conn	Lucas Dev					@October 13, 2025		Urban AI

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
ect - Design								
Estudo de Pricing						@July 23, 2025 → July 25, 2025		Urban AI
Monitorar propriedades por coordenada de eventos								Urban AI
Verificações e Mensageria								Urban AI
Refatoração tela settings	Lucas Dev					@September 30, 2025		Urban AI
Excluir propriedades	Lucas Dev					@September 30, 2025		Urban AI
Alterar rário padrão de 25 para 30	Lucas Dev					@September 30, 2025		Urban AI
Painel de notificações								Urban AI
Agendador de notifi								Urban AI

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
cações								
Listagem propriedades / eventos								Urban AI
Fazer pull request feat/mailsender	Lucas Dev					@September 10, 2025		Urban AI
Integração com serviço de e-mails								Urban AI
Registro de propriedades								Urban AI
Urban Infra	Humphry Davy						High	Urban AI
Engine de proximidade	Jefferson Severino					@July 18, 2025		Urban AI
Crawlers/Coleta de eventos	Marcos Gabriel					@July 18, 2025		Urban AI
Front-end para menu "Eventos"	Lucas Dev			100				Urban AI
Back-end para	Lucas Dev					@July 29, 2025		Urban AI

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
menu "Eventos"								
Erro prisma na AWS	Lucas Dev					@July 31, 2025		Urban AI
Empoli - String	Lucas Dev					@August 12, 2025		Urban AI
Urban - Front Mapa interativo (mock)	Lucas Dev			100		@August 12, 2025		Urban AI
Urban - Stripe	Lucas Dev					@August 12, 2025		Urban AI
Salvar propriedade - Back-end	Lucas Dev					@August 6, 2025		Urban AI
Salvar propriedade - Front-end	Lucas Dev					@July 31, 2025		Urban AI
Notificação de Proximidade de Eventos para Usuários - Back end	Lucas Dev			100		@July 29, 2025		Urban AI
Esquecer senha e confi	Lucas Dev					@August 14, 2025		Urban AI

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
rmação								
Feature pricing - Back-end	Lucas Dev					@August 15, 2025		Urban AI
Integrações backend	Jefferson Severino							Urban AI
Dashboard com mapa								Urban AI
Alter table	Jefferson Severiano					@June 6, 2025		Urban AI
Integração com Airbnb API	Jefferson Severino					@May 30, 2025		Urban AI
Autenticação de usuários								Urban AI
ferramenta multilingue	Jefferson Severino					@May 13, 2025		Urban AI
ajuste logo	Jefferson Severino					@May 9, 2025		Urban AI
Criando ambiente	Jefferson Severino					@May 9, 2025		Urban AI
Layouts iniciais	Lucas Dev					@May 9, 2025		Urban AI
Refino UX	Lumina Lab					@May 6, 2025		Urban AI

Task name	Assignee	Blocked by	Blocking	Completion	Completion Date	Due Date	Priority	Projects
Setup do Projeto	Jefferson Severino					@April 29, 2025		Urban AI
Definição de arquitetura						@April 25, 2025		Urban AI
Testes manuais e QA								Urban AI
Deploy Urban / Bug	Lucas Dev Jefferson Severiano Jefferson Severino							Urban AI

Notificação de eventos por e-mail com verificação de envio